

Prácticas de PSyD

Daniel Báscones García

24 de julio de 2026

Índice

1. Configuración del entorno de programación	3
1.1. Paso 1: Configuración para máquina virtual	3
1.2. Paso 2: Creación de la Máquina Virtual	3
1.3. Paso 3: Configuración del ToolKit de STM	4
1.4. Paso 4: Descarga de manuales y recursos extra	4
2. Práctica 1: Periféricos mapeados en memoria (MMIO): Entrada/Salida de propósito general (GPIO)	6
2.1. Objetivos	6
2.2. Fundamentos teóricos	6
2.3. Desarrollo de la práctica	9
2.3.1. Creación del proyecto	9
2.3.2. Depuración del proyecto	12
2.3.3. Desarrollo del proyecto	15
2.4. Para hacer más	16
3. Práctica 2: Creación de un <i>Board Support Package</i> (BSP)	17
3.1. Objetivos	17
3.2. Fundamentos teóricos	17
3.2.1. Definiciones de direcciones	17
3.2.2. Definiciones de registros	18
3.2.3. Gestión de registros	19
3.2.4. Funciones de control de registros	19
3.2.5. Gestión directa bit a bit de registros	20
3.3. Desarrollo de la práctica	21
3.4. Para hacer más	22
4. Práctica 3: Entrada / Salida programada - UART TX	23
4.1. Objetivos	23
4.2. Fundamentos teóricos	23
4.2.1. UART	23
4.2.2. UART en el STM32F723IE	24
4.3. Desarrollo de la práctica	24
4.4. Para hacer más	26
5. Práctica 4: Entrada / Salida por interrupciones - UART RX y botones	27
5.1. Objetivos	27
5.2. Fundamentos teóricos	27
5.2.1. Configuración de las interrupciones en el procesador	27
5.2.2. Gestión de interrupciones mediante funciones	28

5.3. Desarrollo de la práctica	28
5.4. Para hacer más	32
6. Práctica 5: Configuración de Timers	33
6.1. Objetivos	33
6.2. Fundamentos teóricos	33
6.3. Desarrollo de la práctica	34
6.4. Para hacer más	37

1. Configuración del entorno de programación

Las prácticas de la asignatura de PSyD se desarrollan en un entorno ya preparado dentro de una máquina virtual. Esto es debido a la comodidad y portabilidad que ofrece, ya que con trasladar un simple fichero, cualquiera puede tener todas las herramientas disponibles en su ordenador. No obstante, también se ofrece aquí una guía de instalación para quien quiera instalarlo todo de manera nativa en su ordenador. La guía se ha realizado para Ubuntu. Su adaptación a otros sistemas operativos (MacOS, o Windows) es relativamente directa, al estar las herramientas disponibles para todas las plataformas.

1.1. Paso 1: Configuración para máquina virtual

El software y los recursos necesarios ya están preinstalados en una máquina virtual basada en **Debian 13.6**. Cuenta con un entorno ligero (gestor de ventanas, navegador básico) y las **VirtualBox Guest Additions** para habilitar el passthrough de dispositivos USB (necesario para conectar la placa).

Puedes descargar la máquina virtual ya configurada y lista para importar en VirtualBox desde <https://drive.google.com/file/d/1U7Qvnkb-LfkJrcboyH4Jzs9iNczMntQg/view?usp=drivesdk>. También es necesario descargar **VirtualBox** para arrancarla. Para ello sigue las siguientes instrucciones:

- Asegúrate de tener instalado **VirtualBox** en su versión más reciente. Puedes descargarlo desde <https://www.virtualbox.org/wiki/Downloads>.
- Importante también descargar, desde el mismo enlace, el “VirtualBox Extension Pack”. Sin él, perderemos funcionalidades de la MV, entre otros, la capacidad de conectarle USBs (como el depurador ST-Link de la placa).

1.2. Paso 2: Creación de la Máquina Virtual

Sigue esta parte sólo si quieres crear una máquina virtual propia desde cero. Si quieres hacerlo de manera nativa, salta a Apartado 1.3.

Crea una nueva máquina virtual e instala **Debian** en su última versión (<https://www.debian.org/index.es.html>) o **Ubuntu** (recomendado versión **24.04 LTS**, descargable desde <https://ubuntu.com/download>). Configura el hardware asignándole los siguientes recursos recomendados:

- **Memoria RAM:** 4096 MB
- **Procesadores:** 2 núcleos (Cores).
- **Memoria de Vídeo:** 128 MB.
- **Tamaño del disco:** 60GB.
- **Tipo de instalación:** Manual. No utilizar la automática ya que instala cosas de más.

Una vez seleccionados los recursos, lanza la máquina y el instalador comenzará automáticamente. Deberás seleccionar cuestiones como el nombre de la máquina, usuario, contraseña... elige el que más te guste. El resto de la configuración se puede dejar por defecto. Ten en cuenta que el proceso de instalación puede tardar una media hora (dependiendo del sistema anfitrión). Se recomienda utilizar las instalaciones gráficas para tener todo el sistema operativo bien preconfigurado. No obstante, también se pueden hacer instalaciones manuales para ajustar mejor las herramientas disponibles y, con ello, el tamaño de la máquina.

Una vez arranque la máquina, es importante instalar los extras de invitado (“Guest additions”), para conseguir que anfitrión y host se comuniquen adecuadamente (y puedan por ejemplo compartir portapapeles, ajustar el tamaño de pantalla, etc). Para ello haz click en **devices > insert guest additions CD image**. Esto cargará un disco en la MV (panel izquierdo). Abre una terminal en la ruta donde se monta el CD (normalmente /media/cdrom) y ejecuta:

```
1 # Actualizar paquetes existentes
2 > sudo apt update && sudo apt upgrade
3 # Instalar herramientas para compilar las guest additions
4 > sudo apt install build-essential dkms linux-headers-$
5 # Instalar guest additions
6 > sudo ./VBoxLinuxAdditions.run
```

1.3. Paso 3: Configuración del ToolKit de STM

Independientemente de si quieres utilizar tu propio sistema operativo para instalar las herramientas, o instalarlas en la máquina virtual, se hace de la misma manera. A continuación se muestran todas las herramientas, repositorios, etc necesarios para tener todo lo necesario en la asignatura. Si prefieres no realizar la instalación, descarga la máquina virtual ya configurada como se indica en el primer paso.

- Instalar STM32CubeIDE (la herramienta principal de desarrollo) desde aquí: <https://www.st.com/en/development-tools/stm32cubeide.html>
- Instalar STM32CubeMX (para configuración avanzada) desde aquí: <https://www.st.com/en/development-tools/stm32cubemx.html>
- Instalar STM32CubeProgrammer (para compatibilidad con zephyr) desde aquí: <https://www.st.com/en/development-tools/stm32cubeprog.html>
- Instalar picocom para comunicar con la placa con posterioridad:

```
1 > sudo apt-get install picocom
```

1.4. Paso 4: Descarga de manuales y recursos extra

Dentro de la MV ya creada, en la carpeta **Documentos** del entorno se encuentran varios recursos en forma de manuales. Si se está haciendo la instalación manual, estos son los recursos que ahí se encuentran y haría falta descargar:

- **rm0431** : https://www.st.com/resource/en/reference_manual/rm0431-stm32f72xxx-and-stm32f73xxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

Manual principal de referencia. Detalla toda la funcionalidad de los procesadores de la serie **STM32F723**, incluyendo periféricos, mapa de memoria, registros, modos de uso, etc.

- **um2140** : https://www.st.com/resource/en/user_manual/um2140-discovery-kit-with-stm32f723ie-mcu-stmicroelectronics.pdf

Manual con la descripción de conexiones y *pinout* detallado del kit de desarrollo **32F723EDISCOVERY**.

- **ds11853** : <https://www.st.com/resource/en/datasheet/stm32f722ic.pdf>

Funcionalidad genérica, especificaciones físicas y características electrónicas del procesador **STM32F723IE**.

- **pm0253** : https://www.st.com/resource/en/programming_manual/pm0253-stm32f7-series-and-stm32h7-series-cortexm7-processor-programming-manual-stmicroelectronics.pdf

Manual de programación de la serie **Cortex-M7**, con la referencia completa del conjunto de instrucciones ensamblador.

- **DB3105**: https://www.st.com/resource/en/data_brief/32f723ediscovery.pdf

Resumen de características del kit de desarrollo **32F723EDISCOVERY**.

Adicionalmente, también se incluye en la MV creada la colección de ejemplos oficiales **STM32CubeF7**.

Nota: Esta colección utiliza librerías de alto nivel (HAL/LL). En la primera parte de las prácticas programaremos “desde cero” (a nivel de registro), por lo que no usaremos estas librerías directamente. Sin embargo, los ejemplos son un buen punto de referencia para explorar las capacidades de la placa.

- **STM32CubeF7 (GitHub)** <https://github.com/STMicroelectronics/STM32Cubef7>

Colección de ejemplos base. Incluye proyectos para múltiples placas, pero nos centraremos únicamente en la **32F723EDISCOVERY**.

Al clonar manualmente el repositorio, es imprescindible inicializar los submódulos para incluir todas las librerías requeridas. Mira bien las instrucciones del repositorio.

- **STM32CubeF7 Patch** <https://www.st.com/en/embedded-software/stm32cubef7.html>

Parque que se debe sobrescribir en el repositorio una vez clonado. Añade librerías propietarias de terceros necesarias para funciones como el control táctil o el procesamiento de audio.

2. Práctica 1: Periféricos mapeados en memoria (MMIO): Entrada/Salida de propósito general (GPIO)

2.1. Objetivos

- Comprender el concepto de **Memory-Mapped I/O (MMIO)**.
- Analizar la organización del mapa de memoria del microcontrolador **STM32F723IE**.
- Aprender a manipular directamente los registros de entrada/salida mediante máscaras de bits en C.

2.2. Fundamentos teóricos

En un procesador de propósito general, numerosas funciones se realizan a través de *periféricos*, componentes anexos al procesador con funciones específicas.

En la arquitectura ARM, los periféricos no se controlan con instrucciones dedicadas, sino que se comunican con la CPU a través del bus del sistema (*AXI-AHB Bus Matrix*) asignándoles direcciones físicas dentro del mapa de memoria de 32 bits (4GB).

Cada periférico posee un conjunto de **registros de control, estado y datos** ubicados en direcciones consecutivas a partir de una dirección base (*Base Address*). Escribiendo y leyendo en esas direcciones, conseguimos comunicarnos con los periféricos, que a su vez se conectarán con los diferentes pines del procesador. Los pines servirán para su conexión con el exterior con otros elementos, como LEDs, botones, pantallas, puertos, etc.

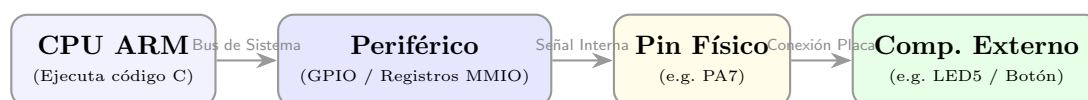


Figura 1: Flujo de control desde el código en CPU hasta el componente exterior a través de MMIO.

En nuestro caso, disponemos de un procesador **STM32F723IE**. Podemos consultar el manual de referencia [rm0431](#) para ver una lista completa de sus periféricos y las direcciones de memoria donde se ubican sus registros. La sección [rm0431 1.6](#) detalla todo este mapa de memoria, que se muestra en las Figuras 2 y 3

De aquí ya podemos deducir las direcciones de los diferentes controladores de GPIO. Vemos que están nombrados con letras (desde GPIOA hasta GPIOI). Por ejemplo, el controlador de GPIOA está en la dirección `0x40020000`. ¿Pero claro, cómo lo manejamos? Para ello debemos conocer todos los registros que se encuentran en esa zona de memoria. Esa información la tenemos en [rm0431 6.4](#).

A modo de ejemplo, se muestra en la Figura 4 la información referente al registro **MODER** del controlador de GPIO. Observamos que, en una palabra de 32 bits, contiene la configuración de modo de pin (Input/Output/Función alternativa/Analógico) para hasta 16 pines diferentes. Es decir, utiliza 2 bits para la configuración de cada pin. Los dos bits menos significativos corresponden al pin 0, y los 2 más significativos al pin 15, con el resto entre medias. Escribiendo en este registro configuraremos el modo de cada uno de estos pines.

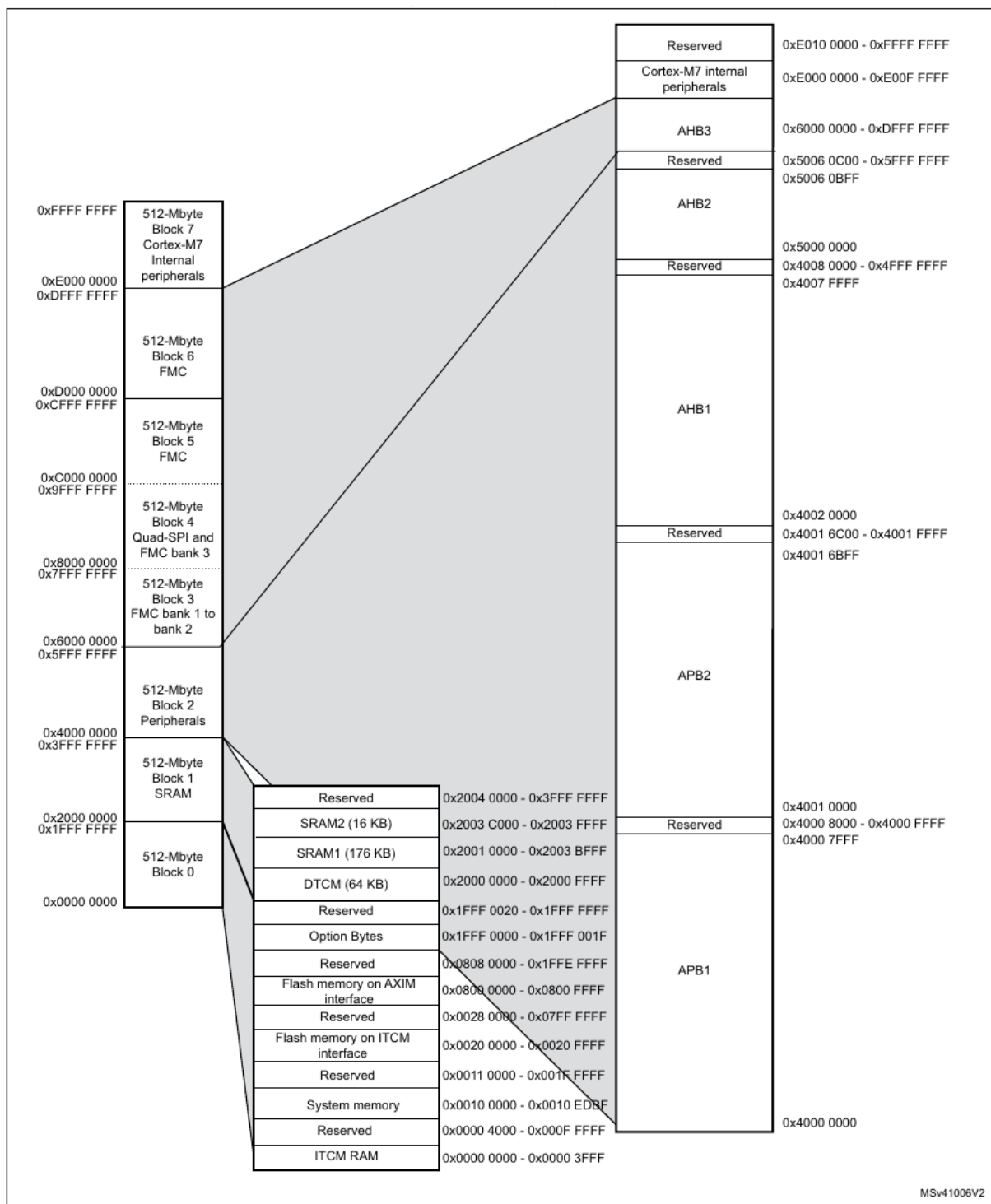


Figura 2: Mapa de memoria del **STM32F723IE**

Table 1. STM32F72xxx and STM32F73xxx register boundary addresses

Boundary address	Peripheral	Bus	Register map
0xA000 1000 - 0xA0001FFF	QuadSPI Control Register	AHB3	Section 13.5.14: QUADSPI register map on page 378
0xA000 0000 - 0xA000 0FFF	FMC control register		Section 12.8.6: FMC register map on page 349
0x5006 0800 - 0x5006 0BFF	RNG	AHB2	Section 16.7.4: RNG register map on page 458
0x5006 0000 - 0x5006 03FF	AES		Section 17.7.18: AES register map on page 510
0x5000 0000 - 0x5003 FFFF	USB OTG FS		Section 32.15.64: OTG_FS/OTG_HS register map on page 1288
0x4004 0000 - 0x4007 FFFF	USB OTG HS	AHB1	Section 32.15.64: OTG_FS/OTG_HS register map on page 1288
0x4002 6400 - 0x4002 67FF	DMA2		Section 8.5.11: DMA register map on page 252
0x4002 6000 - 0x4002 63FF	DMA1		
0x4002 4000 - 0x4002 4FFF	BKPSRAM		Section 5.3.27: RCC register map on page 192
0x4002 3C00 - 0x4002 3FFF	Flash interface register		Section 3.7.9: Flash interface register map
0x4002 3800 - 0x4002 3BFF	RCC		Section 5.3.27: RCC register map on page 192
0x4002 3000 - 0x4002 33FF	CRC		Section 11.4.6: CRC register map on page 275
0x4002 2000 - 0x4002 23FF	GPIOI		Section 6.4.11: GPIO register map on page 212
0x4002 1C00 - 0x4002 1FFF	GPIOH		
0x4002 1800 - 0x4002 1BFF	GPIOG		
0x4002 1400 - 0x4002 17FF	GPIOF		
0x4002 1000 - 0x4002 13FF	GPIOE		
0x4002 0C00 - 0x4002 0FFF	GPIOD		
0x4002 0800 - 0x4002 0BFF	GPIOC		
0x4002 0400 - 0x4002 07FF	GPIOB		
0x4002 0000 - 0x4002 03FF	GPIOA		

Figura 3: Detalle del mapa de memoria del **STM32F723IE**

6.4.1 GPIO port mode register (GPIOx_MODER) (x = A to K)

Address offset: 0x00

Reset value: 0xA800 0000 for port A

Reset value: 0x0000 0280 for port B

Reset value: 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODER[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Figura 4: Detalle del registro MODER

Hemos visto pues que podemos configurar ciertos puertos de entrada/salida, que a su vez cuentan con numerosos pines. Pero, ¿en qué puerto/pin se sitúa, por ejemplo, un LED?. Para ello, debemos acudir a la documentación de la placa, donde se han diseñado físicamente las conexiones desde el procesador, hasta los diferentes elementos de soporte. En concreto, en la sección [um2140 5.16](#) tenemos la lista de todas las conexiones de botones y LEDs. Lo vemos en la Figura 5

Finalmente, sabemos que, por ejemplo, el LED5 se controla desde el puerto GPIOA, en el PIN 7. El puerto A está en la dirección [0x40020000](#), y dispone de varios registros que deberemos configurar adecuadamente para controlar, finalmente, el LED.

2.3. Desarrollo de la práctica

2.3.1. Creación del proyecto

Vamos a controlar, directamente y escribiendo en registros del procesador, un LED (en concreto, el LED5 que acabamos de ver). Pero antes, debemos crear un proyecto en STMCubeIDE, la herramienta de desarrollo de ST. Para ello:

1. Abrimos STM32CubeIDE, buscándolo en el menú del sistema. Elegimos un workspace (o carpeta de trabajo) donde trabajar. Tras abrirlo, veremos el programa de la Figura 6
2. Creamos un nuevo proyecto con **File >STM32 Project Create / Import**.
3. Seleccionamos **STM32CubeIDE Empty Project** (Figura 7)
4. En **Board Selector**, buscar 723e y seleccionar la única placa disponible (Figura 8).

Table 5. Control port assignment

Reference	Color	Name	Comment
B1	BLUE	USER	Alternate function Wake-up
B2	BLACK	RESET	-
LD1	BLUE	ARDUINO	PA5
LD2	RED	5 V Power	-
LD3	RED	Fault Power	Current upper than 625 mA
LD4	RED/GREEN	ST-LINK COM	Green during communication
LD5	RED	USER1	PA7
LD6	GREEN	USER2	PB1
LD7	RED	USB OTG HS OVCR	PH10
LD8	GREEN	V _{BUS} USB HS	PB13
LD9	RED	USB OTG FS OVCR	PB10
LD10	GREEN	V _{BUS} USB FS	PA9

Figura 5: Pines para LEDS y botones de la placa **32F723EDISCOVERY**

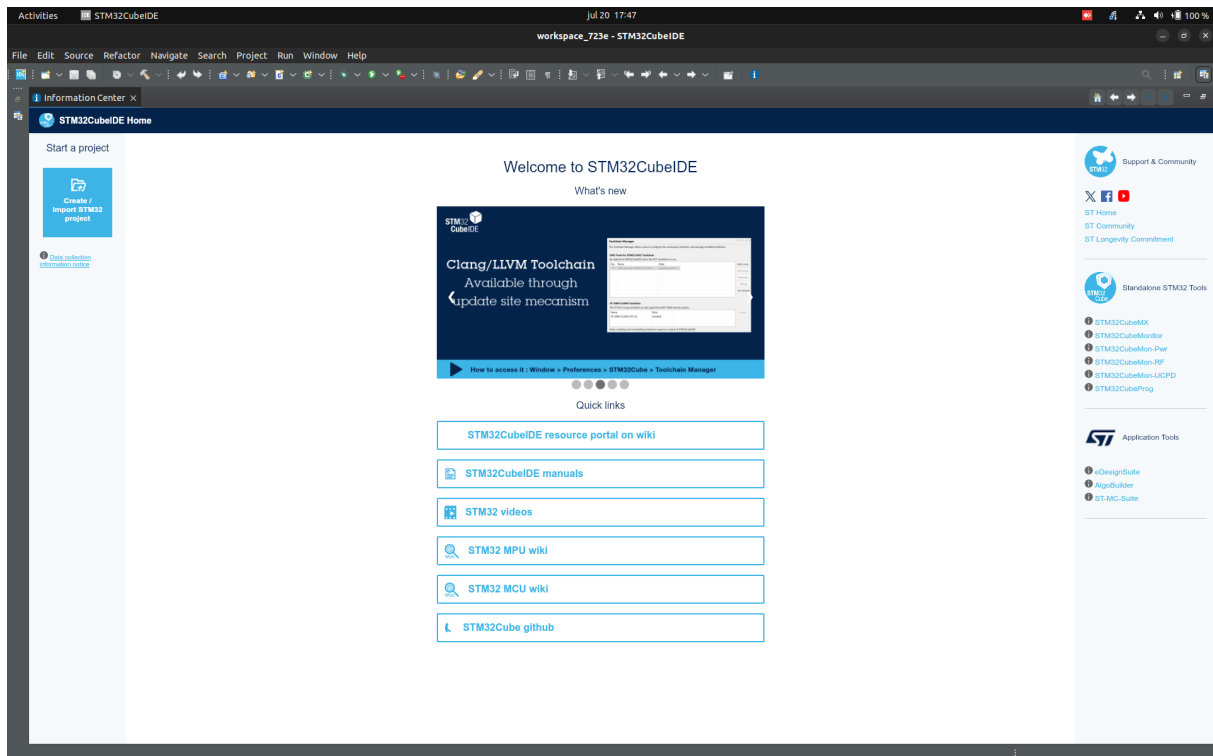


Figura 6: Ventana inicial de la aplicación

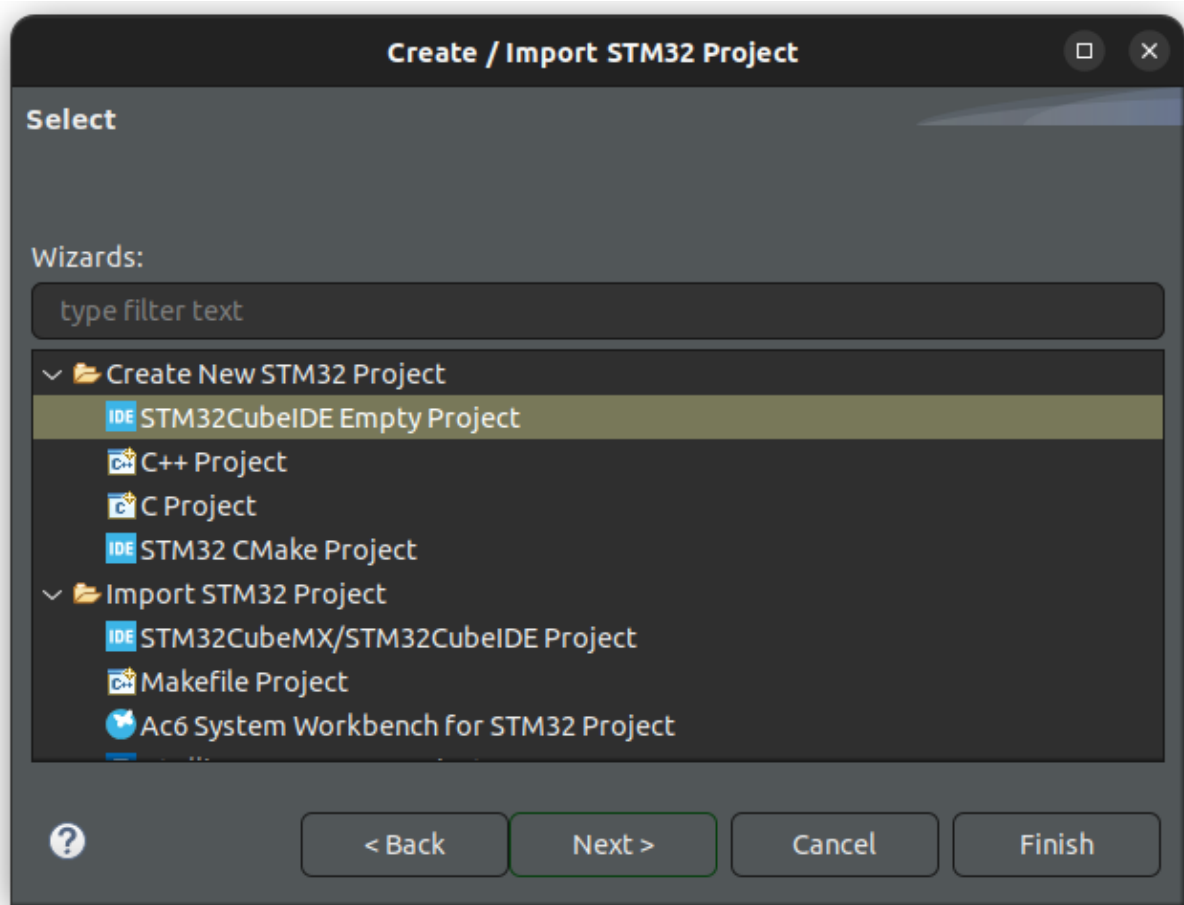


Figura 7: Creación de proyecto

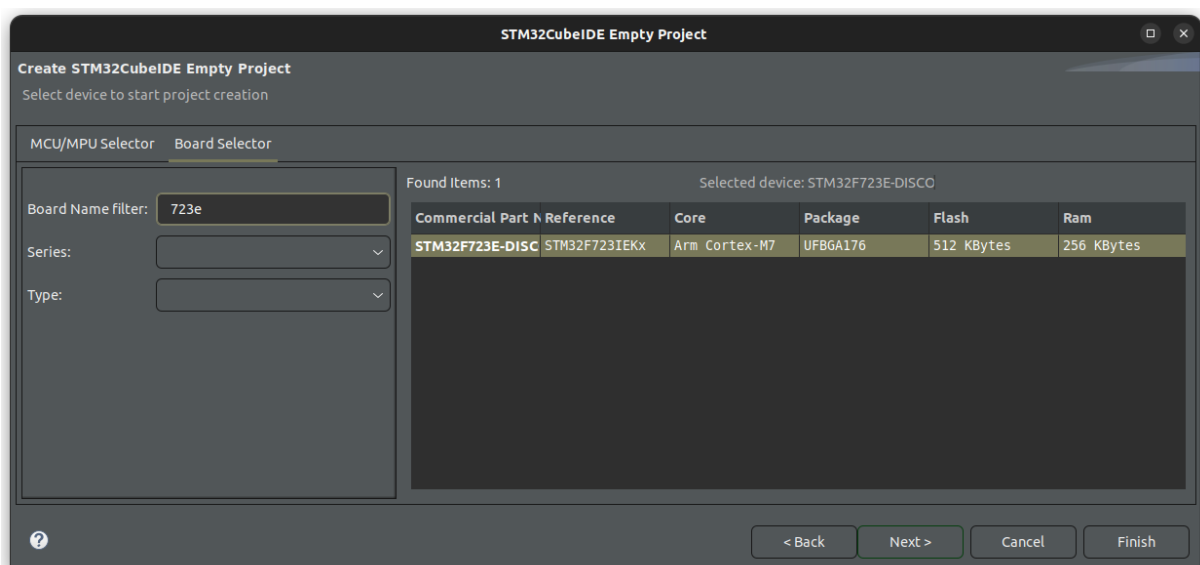


Figura 8: Selección de placa

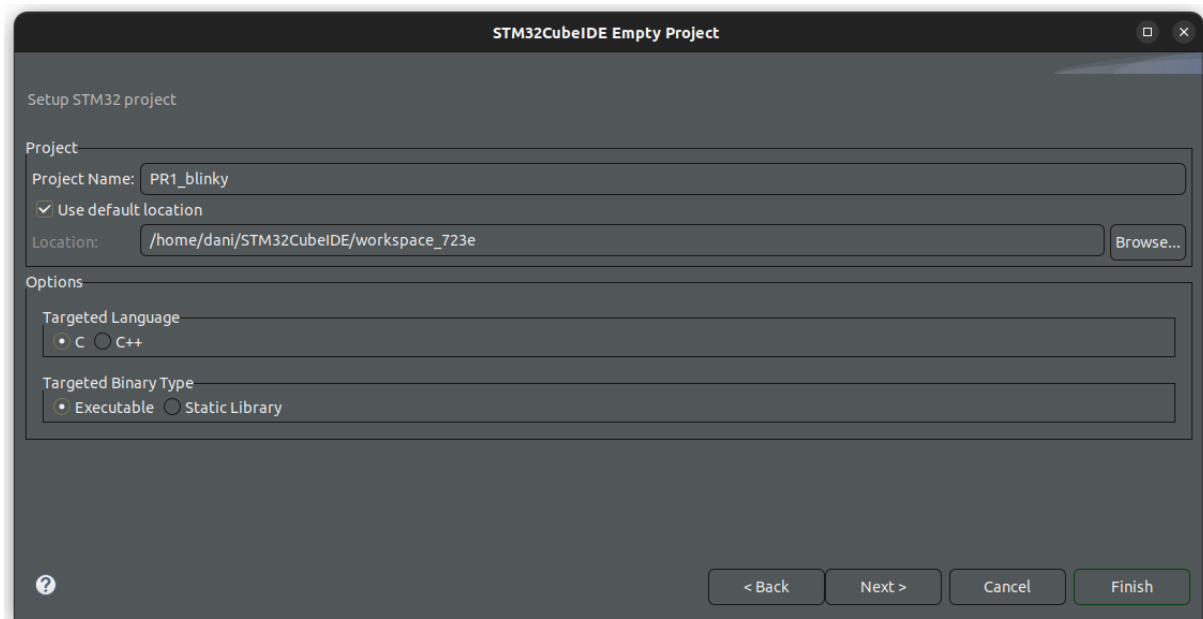


Figura 9: Selección de nombre de proyecto

5. Finalmente, dar un nombre al proyecto y terminar (Figura 9).

2.3.2. Depuración del proyecto

Ahora deberíamos tener un proyecto en blanco para trabajar (Figura 10). Antes de nada, vamos a probar que la conexión con la placa funciona, y podemos compilar y depurar el proyecto. Para ello:

1. Haz doble click en la carpeta **Src** dentro de tu proyecto, y luego, en **main.c**.
2. Con el archivo abierto, haz click en el martillo (build) que se encuentra en la barra superior de tareas. Verás el log de compilación en la terminal (Figura 11).
3. Finalmente, y si no ha habido problemas, haz click en el icono de depuración (debug) en la misma barra superior (el bichito verde).
4. Acepta la configuración de depuración por defecto (Figura 12) y vuelve a aceptar cuando pida cambiar a la vista de depuración.

Ahora puedes depurar el proyecto poco a poco. Entre otras, el depurador permite:

- Ir paso a paso por las líneas de código.
- Colocar *breakpoints* para parar la ejecución al llegar a ciertas líneas.
- Inspeccionar el contenido de los registros del procesador.
- Inspeccionar el contenido de los registros de los diferentes periféricos.
- Inspeccionar el contenido de la memoria.

Se recomienda explorar todas estas funcionalidades, que se pueden ver en la Figura 13.

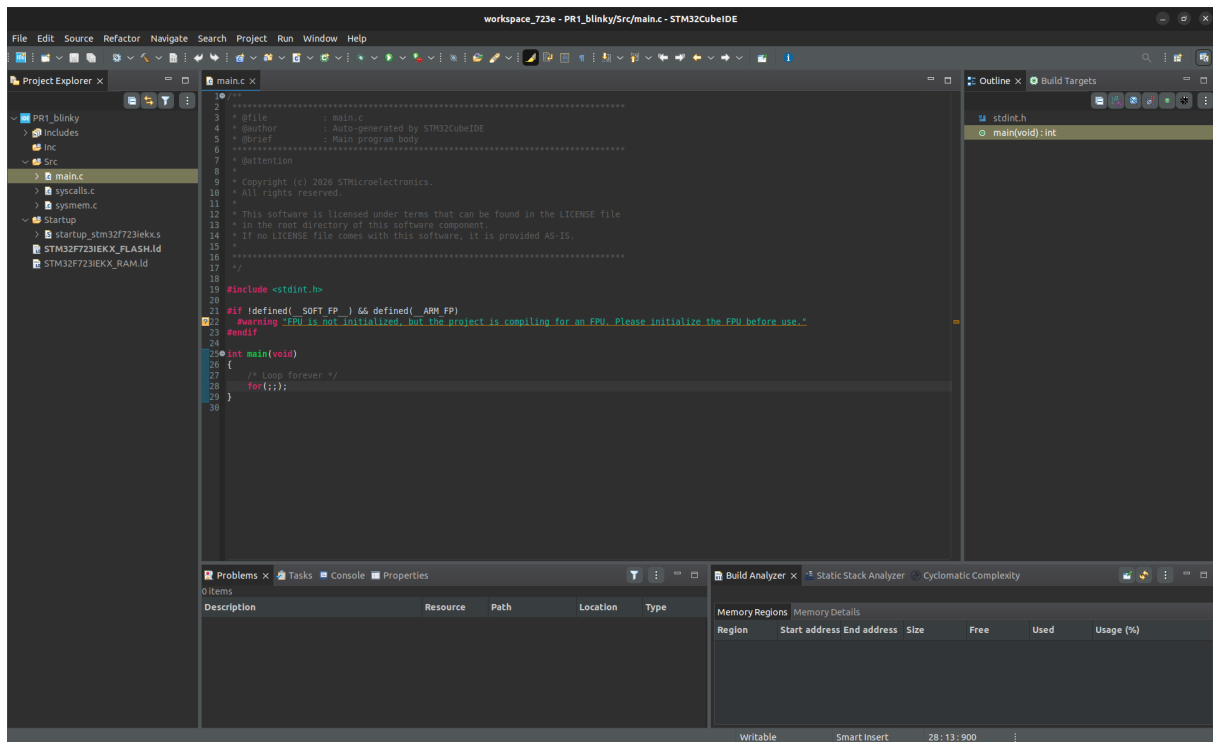


Figura 10: Aplicación por defecto

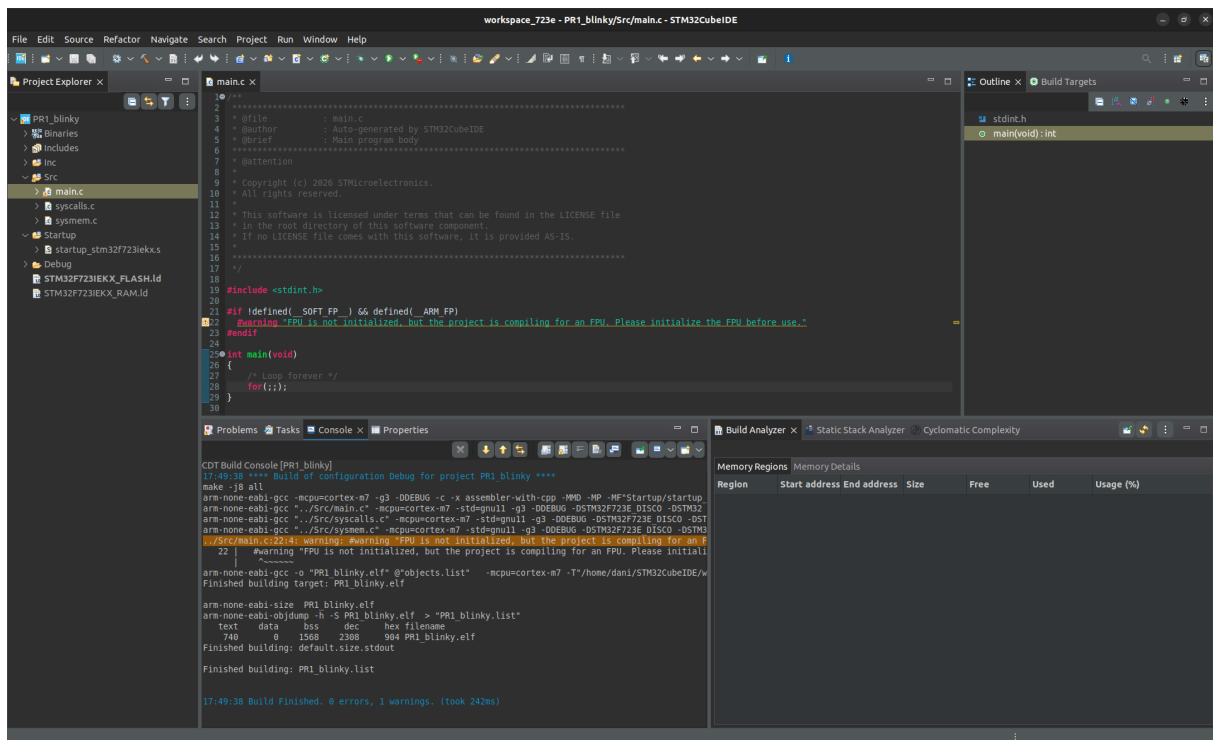


Figura 11: Log tras construir la aplicación

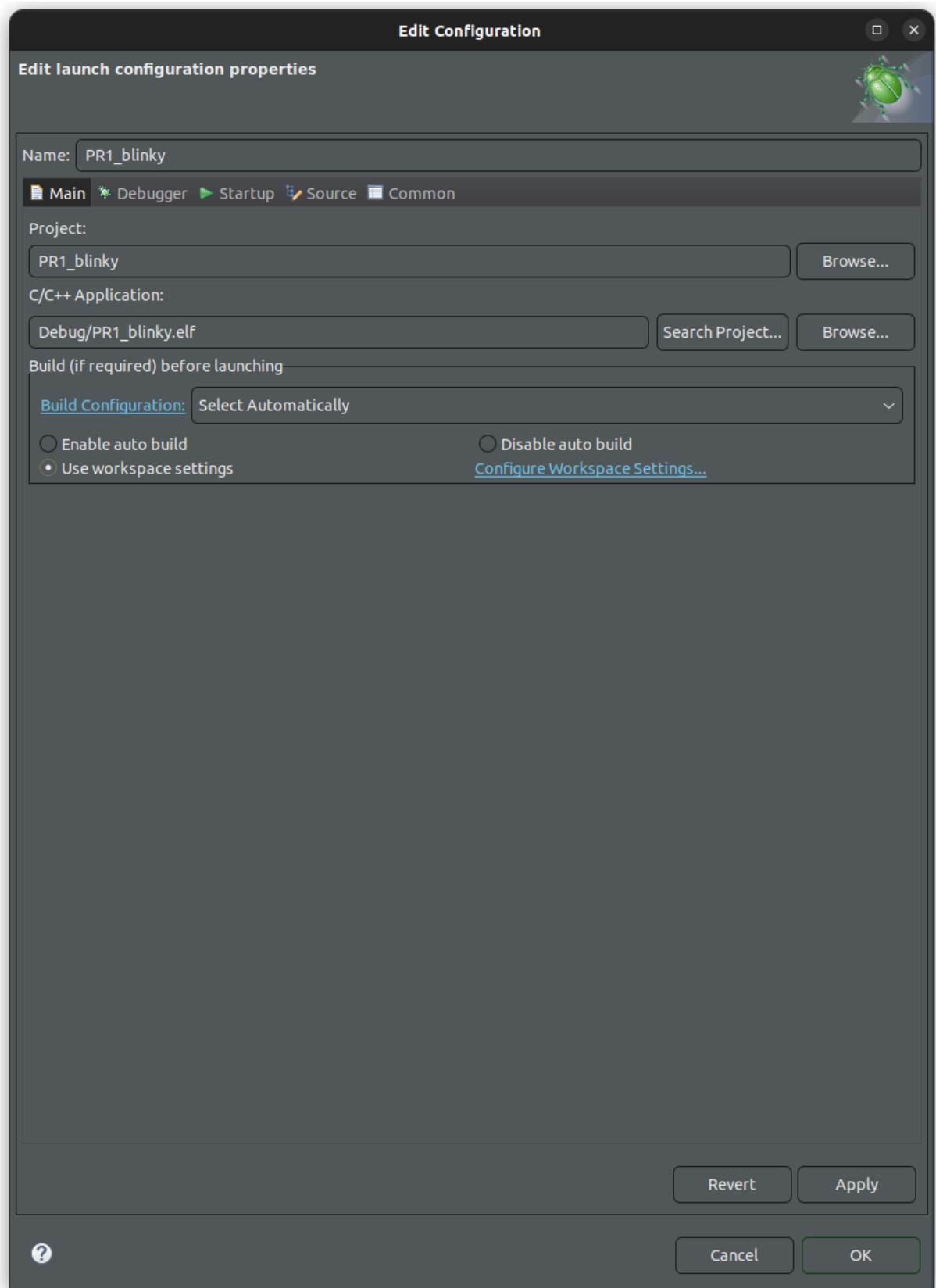


Figura 12: Selección de depuración por defecto

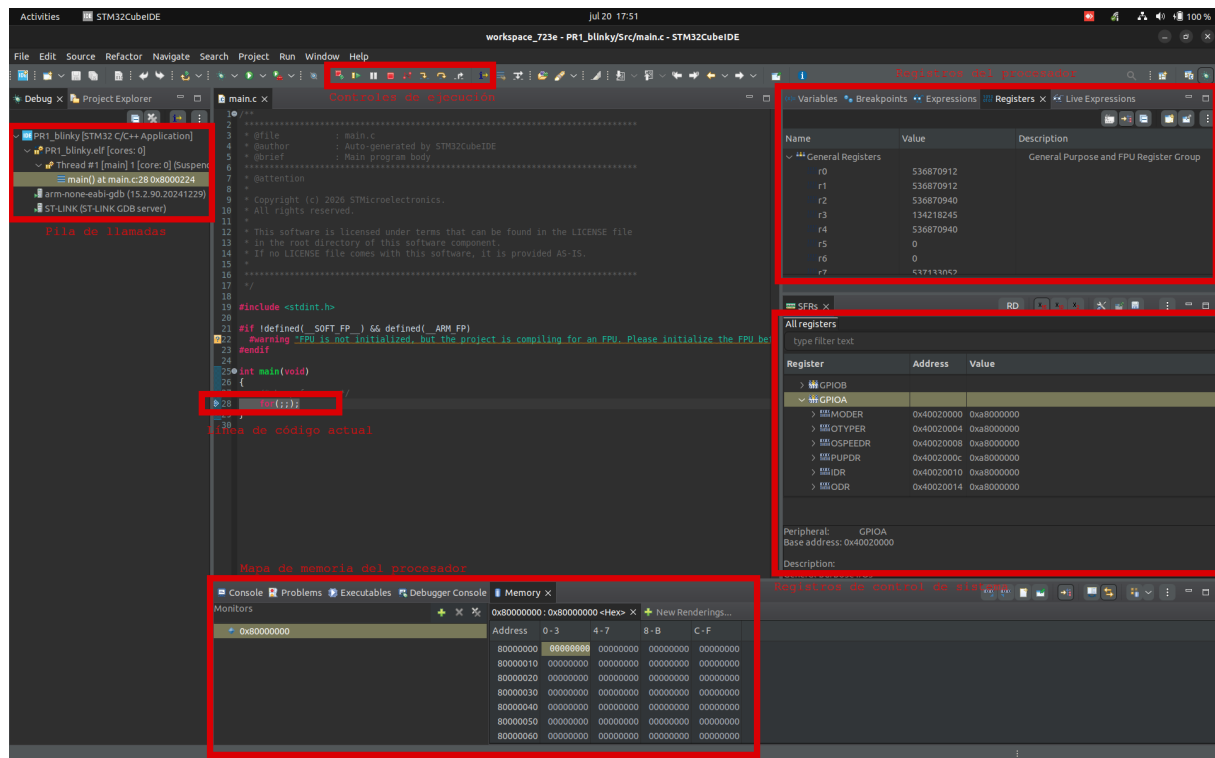


Figura 13: Ventana de depuración personalizada

2.3.3. Desarrollo del proyecto

Finalmente, debemos programar nuestro código para encender el LED. Para ello, habrá que escribir en las direcciones concretas de los registros de GPIO, en este caso de GPIOA-PIN7. Recuerda que la documentación de los registros disponibles está en [rm0431 6.4](#).

- Configurar GPIOA-PIN7 como salida en **MODER**.
- Poner el pin en tipo *push-pull* en **OTYPER**.
- Configurar la velocidad a alta en **OSPEEDR**.
- Deshabilitar las resistencias en **PUPDR**.
- Finalmente, se podrá escribir el valor del led (0/1) a través de **ODR**. Otra opción es escribir directamente al registro **BSRR**, que tiene la ventaja de evitar lecturas o escrituras múltiples (aunque para nuestro caso no afecta en nada).

Adicionalmente, tenemos que activar el reloj del periférico GPIOA a través del registro **RCC_AHB1ENR** ([rm0431 5.3.10](#)). La dirección base la podemos consultar en [rm0431 1.6.2](#).

Quizá te quede la pregunta de cómo narices escribo yo en una dirección de memoria concreta... la respuesta son... ¡punteros!

```
1 //Imaginemos que quiero escribir en la dirección 0x40020000
2 //justo en los bits 14-15
3 // (por lo que sea)
4
5 //borro configuración actual (máscara con ceros en las posiciones de interés)
```

```

6 * ((uint32_t *) (0x40020000)) &= ~(0b11 << (7*2));
7 //pongo la configuración que me interesa
8 * ((uint32_t *) (0x40020000)) |= 0b01 << (7*2);

```

Aprovecha también esta función para hacer que parpadee cada cierto tiempo:

```

1 void delay(int loops) {
2     for (int i = 0; i < loops; i++) {
3         __asm("nop");
4     }
5 }

```

Nuestra función main.c tendrá por tanto esta pinta:

```

1 int main(void)
2 {
3     //activar reloj de GPIOA en RCC_AHB1ENR
4     //configurar Pin0-GPIOA como salida
5     //configurar Pin0-GPIOA modo push-pull
6     //configurar Pin0-GPIOA velocidad alta
7     //configurar Pin0-GPIOA con pullup/down desactivado
8
9     /* Loop forever */
10    for(;;) {
11        //encender LED
12        delay(1000000);
13        //apagar LED
14        delay(1000000);
15    }
16 }

```

2.4. Para hacer más

Busca el GPIO del botón de la placa, configúralo en modo entrada, y haz que el LED se encienda o apague cada vez que se pulse el botón, en lugar de hacerlo de manera automática.

3. Práctica 2: Creación de un *Board Support Package* (BSP)

3.1. Objetivos

- Desarrollar un *Board Support Package* minimalista.
- Entender la importancia de crear, en la medida de lo posible, código no dependiente del hardware.

3.2. Fundamentos teóricos

Un *Board Support Package* o BSP comprende una colección de bibliotecas que ayudan al programador a trabajar con una placa o procesador determinado. Generalmente, incluyen definiciones, funciones, macros y demás utilidades para que el programador no tenga que trabajar directamente con código de muy bajo nivel como, por ejemplo, hemos hecho en la práctica anterior. Por ejemplo:

```
1 // En lugar de un código así
2 * ((uint32_t *) (0x40020000)) &= ~(0b11 << (7*2));
3 * ((uint32_t *) (0x40020000)) |= 0b01 << (7*2);
4
5 // Buscamos algo así
6 GPIO_SET_MODE(GPIOA, PIN7, MODE_OUTPUT);
```

Por supuesto, alguien tiene que ser el encargado de crear, por primera vez, este BSP. En sistemas empuotrados, normalmente el BSP tiene que ser creado a medida, pues la combinación de procesador, periféricos y pines es particular para cada placa desarrollada. Para la placa **32F723EDISCOVERY**, ya existe un BSP, y se anima al alumno a consultarlo y ver cómo está construido. A lo largo de estas prácticas, desarrollaremos otro, mucho más minimalista y comedido, que aporte la funcionalidad justa que necesitamos. En esta práctica comenzamos con ese trabajo.

Vamos a ver el ejemplo concreto de cómo desarrollar la función anterior, para entender mejor las partes que puede tener un BSP.

3.2.1. Definiciones de direcciones

En primer lugar, necesitamos definir las direcciones de memoria donde se encuentran todos nuestros periféricos. Esto cumple una doble función: Por un lado, si cambiaran, las podemos modificar fácilmente al estar definidas como macros. Por otro lado, hacemos el código reutilizable para procesadores de la misma familia (por ejemplo, todos los Cortex-M7 de ARM (como el **STM32F723IE** que utilizamos) reutilizan partes de los mismos BSPs).

```
1 #define GPIO_BASE_ADDR 0x40020000
2 #define GPIOA_BASE_ADDR (GPIO_BASE_ADDR + 0x0000)
3 #define GPIOB_BASE_ADDR (GPIO_BASE_ADDR + 0x0400)
4 ...
```

Table 24. GPIO register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOA_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00	GPIOB_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0
0x00	GPIOx_MODER (where x = C..K)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	GPIOx_OTYPER (where x = A to K)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figura 14: Extracto parcial de registros del controlador de GPIO

3.2.2. Definiciones de registros

Lo siguiente es definir los registros que se encuentran en dichas direcciones. Como suelen ser varios, y consecutivos, se suele definir una estructura de tipo (en lenguaje C) que los englobe todos, por facilidad de uso. Por ejemplo, mirando [rm0431 6.4.11](#), podemos ver todos los registros contenidos en cada controlador de GPIO. Se muestra la primera parte de la tabla de registros en la Figura 14

Un detalle interesante a ver aquí es que, por ejemplo, para el registro `MODER`, nos dice que tiene tres valores por defecto (reset) diferentes, según si se trata del puerto A, B, u otros. El registro `OTYPER`, sin embargo, tiene el mismo valor por defecto para todos los puertos (A-K). En cada registro, además, nos indica qué bit controla qué cosa. Para los registros `MODER`, los bits, de dos en dos, controlan el modo de GPIO (especificados en [rm0431 6.4.1](#)). Para `OTYPER`, controla el tipo de salida ([rm0431 6.4.2](#)). Si hacemos el código para tener una estructura que nos permita controlar estos registros, tendríamos algo de este estilo:

```

1 typedef struct
2 {
3     volatile uint32_t MODER;    /*!< GPIO port mode register, Address offset: 0
4     volatile uint32_t OTYPER;  /*!< GPIO port output type register, Address
5     ...                          offset: 0x04
6 } GPIO_TypeDef;

```

Importante observar la declaración de las variables como tipo `volatile`, ya que, además de modificarse internamente desde el procesador, son registros cuyo valor puede modificarse desde fuera (por el propio control del periférico). Con la palabra clave `volatile` conseguimos que el compilador no optimice esas variables y siempre las consulte directamente de memoria.

Pregunta: ¿A qué se debe que todos los registros queden colocados en direcciones sucesivas de memoria al ser declarados con esta sintaxis?

Pregunta: Explora qué hace exactamente el compilador al trabajar con objetos volátiles.

3.2.3. Gestión de registros

Tenemos definidas las direcciones, y las estructuras de registros. Ahora necesitamos tener una forma de escribir en dichas estructuras. Para ello, vamos a crear punteros a las direcciones de memoria base, con el tipo de estructura definido:

```
1 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE_ADDR)
```

Si la base del controlador se encuentra en la dirección `0x40020000` (como es el caso de GPIOA), los registros se encontrarán a continuación (el primero `MODER` en `0x40020000`, el segundo `OTYPER` en `0x40020004`, y así sucesivamente). Así, lo que conseguimos es que, posteriormente, en el código de nuestro programa, podamos hacer algo de este estilo:

```
1 GPIOA->MODER = 0xA8000000;  
2 uint32_t temp = GPIOA->MODER;
```

Exactamente equivalente a haber escrito (o leído) desde las direcciones de memoria correspondientes. Es decir, de una forma legible, acceder (para escribir o leer) los contenidos de estos registros, que a la vez sirven para control de los periféricos.

Pregunta: Cada registro normalmente controla, desde cada uno de sus bits, elementos diferentes del periférico. ¿Cómo puedo leer (o escribir) en unos bits concretos del registro?

3.2.4. Funciones de control de registros

Aunque ahora es más cómodo trabajar con los registros, lo ideal es contar con funciones que nos ayuden, aún más, a abstraer todos estos códigos de bajo nivel que son propensos a errores por parte del programador. Por ejemplo, ahora mismo, si quisiéramos cambiar el modo del pin 7, del puerto A, a *output*, deberíamos hacer algo de este estilo:

```
1 GPIOA->MODER &= ~(0b11 << (7*2));  
2 GPIOA->MODER |= 0b01 << (7*2);
```

Es decir, borramos primero los dos bits de las posiciones 14 y 15 (Ver tabla anterior) y luego les ponemos el valor "01" para configurarlo a nuestro gusto. Pero, ¿y si ahora queremos cambiar el pin 4 del puerto B? ¿o ponerlo en modo *input*?. Es mejor crear una función genérica que nos permita hacer todo.

Aunque esto ya es a gusto del consumidor, la sugerencia para estas funciones es hacerlas lo más genéricas posibles. En este caso, deberíamos poder permitir trabajar con cualquier puerto, cualquier pin, y cualquier modo. Por ejemplo, podemos hacer algo así:

5.3.10 RCC AHB1 peripheral clock register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0010 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OTGHS ULPIEN	OTGHS EN	Res.	Res.	Res.	Res.	Res.	Res.	DMA2 EN	DMA1 EN	DTCMRA MEN	Res.	BKPSR AMEN	Res.	Res.
	rw	rw							rw	rw	rw		rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CRC EN	Res.	Res.	Res.	GPIOI EN	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GIOD EN	GPIOC EN	GPIO BEN	GPIO AEN
			rw				rw	rw	rw	rw	rw	rw	rw	rw	rw

Figura 15: Detalle de bits del registro **AHB1ENR**

```
1 typedef enum {
2     GPIO_MODE_INPUT    = 0x00,
3     GPIO_MODE_OUTPUT   = 0x01,
4     GPIO_MODE_ALT      = 0x02,
5     GPIO_MODE_ANALOG   = 0x03
6 } GPIO_Mode_t;
7
8 static inline void GPIO_MODE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin,
9     GPIO_Mode_t mode) {
10     uint32_t temp = GPIOx->MODER; //leemos registro
11     temp &= ~(0b11 << (pin * 2)); //borramos modo actual
12     temp |= (mode << (pin * 2)); //ponemos modo nuevo
13     GPIOx->MODER = temp; //escribimos de vuelta
14 }
```

Con esto, ya podríamos hacer desde nuestro código principal, lo siguiente:

```
1 GPIO_MODE_SET(GPIOA, 7, GPIO_MODE_OUTPUT);
```

3.2.5. Gestión directa bit a bit de registros

Otros registros, como por ejemplo **AHB1ENR** ([rm0431 5.3.10](#)) tienen una estructura que no es regular, ya que cada bit controla una cosa diferente, como podemos ver en la Figura 15.

Para estos casos, C nos permite una sintaxis como la siguiente, que nos deja acceder individualmente a nivel de bit a los registros:

```
1 //Base address
2 #define RCC_AHB1EN_BASE_ADDR 0x40023830
3
4 //Structure for Reset and Clock Control peripheral clock register 1
5 typedef union {
6     volatile uint32_t reg;
7
8     // 2. Access individual bits or bit-groups for the register above
9     struct {
10         volatile uint32_t GPIOAEN : 1; // Bit 0: Enable peripheral (1 bit
11     )
12 }
```

```

11         volatile uint32_t GPIOBEN      : 1;  // Bit 1: Enable peripheral (1 bit
12         )
13     } bits;
14 } RCC_AHB_ENR_TypeDef;
15
16 //Handle:
17 #define RCC_AHB1ENR      ((RCC_AHB_ENR_TypeDef *) RCC_AHB1EN_BASE_ADDR)

```

Posteriormente, podemos acceder a los campos individuales de la siguiente manera:

```

1 RCC_AHB1ENR->bits.GPIOAEN = 1;

```

Este registro en concreto es muy importante, pues activa los periféricos. De no activarlos, no funcionarán aunque se configuren adecuadamente.

3.3. Desarrollo de la práctica

Crea un BSP para que, en el código de la práctica anterior, no haya que acceder directamente a registros del periférico GPIOA. Necesitarás funciones para configurar **MODER**, **OTYPER**, **OSPEEDR**, **PUPDR** y también para escribir 1 o 0 (esto lo puedes hacer desde el registro **ODR** o **BSRR**). Consulta [rm0431 6.4](#) para ver todos los registros y sus funcionalidades. Deberás:

1. Completar la definición de tipo para incluir todos los registros. También completar la definición de tipo de **AHB1ENR**.

```

1 //... completar direcciones
2 #define GPIO_BASE_ADDR
3 #define GPIOA_BASE_ADDR (GPIO_BASE_ADDR + 0x0000)
4 #define RCC_AHB1EN_BASE_ADDR
5
6 //Structure for GPIO controllers
7 typedef struct
8 {
9     volatile uint32_t MODER;    /*!< GPIO port mode register,
10                                Address offset: 0x00    */
11     volatile uint32_t OTYPER;   /*!< GPIO port output type register,
12                                Address offset: 0x04    */
13     //... completar
14 } GPIO_TypeDef;
15
16 //Structure for Reset and Clock Control peripheral clock register 1
17 typedef union {
18     volatile uint32_t reg;
19
20     // 2. Access individual bits or bit-groups for the register above
21     struct {
22         volatile uint32_t GPIOAEN      : 1;  // Bit 0: Enable peripheral
23         (1 bit)
24         volatile uint32_t GPIOBEN      : 1;  // Bit 1: Enable peripheral
25         (1 bit)
26         //... completar
27     } bits;
28 } RCC_AHB_ENR_TypeDef;
29
30 #define GPIOA      ((GPIO_TypeDef *) GPIOA_BASE_ADDR)
31 #define RCC_AHB1ENR ((RCC_AHB_ENR_TypeDef *) RCC_AHB1EN_BASE_ADDR)

```

2. Crear las funciones, junto con sus definiciones de valores, para controlar los diferentes registros. Toma el ejemplo previo de la función `GPIO_MODE_SET` para hacer las demás.

```
1 static inline void GPIO_MODE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin
  , GPIO_Mode_t mode);
2 static inline void GPIO_OTYPE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t
  pin, GPIO_OType_t otype);
3 static inline void GPIO_OSPEED_SET(volatile GPIO_TypeDef *GPIOx, uint32_t
  pin, GPIO_OSpeed_t ospeed);
4 static inline void GPIO_PUPD_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin
  , GPIO_PuPd_t pupd);
5 static inline void GPIO_PIN_WRITE(volatile GPIO_TypeDef *GPIOx, uint32_t
  pin, GPIO_State_t state);
```

3. Recuerda empaquetar toda esta funcionalidad en un fichero `bsp.h` que irá dentro de la carpeta `Inc` del proyecto.
4. Adaptar el código original `main.c` de la práctica para utilizar el BSP.

```
1 int main(void)
2 {
3     RCC_AHB1ENR->bits.GPIOAEN = 1;
4
5     GPIO_MODE_SET(GPIOA, 7, GPIO_MODE_OUTPUT);
6     //... completar
7
8     for(;;) {
9         GPIO_PIN_WRITE(GPIOA, 7, GPIO_STATE_ONE);
10        delay(1000000);
11        //... completar
12        delay(1000000);
13    }
14 }
```

3.4. Para hacer más

Abre el proyecto BSP de ejemplo (dentro del paquete `STM32CubeF7`) e investiga cómo están organizadas las funciones, definiciones, registros... Para navegar por el proyecto, puedes ir haciendo `ctrl+click` en las llamadas a funciones o variables, y te llevará al fichero donde están definidas.

4. Práctica 3: Entrada / Salida programada - UART TX

4.1. Objetivos

- Comprender más en profundidad el concepto de "periférico" dentro de un procesador.
- Aprender a programar un periférico.
- Interactuar con el periférico desde fuera de la placa.

4.2. Fundamentos teóricos

4.2.1. UART

La comunicación **UART** (*Universal Asynchronous Receiver-Transmitter*) es un protocolo de comunicación serie asíncrono utilizado ampliamente en sistemas embebidos. Al ser asíncrono, no requiere de una señal de reloj compartida entre el emisor y el receptor; en su lugar, ambos dispositivos deben estar configurados previamente a la misma velocidad de transmisión (*baud rate*).

Estructura de la Trama UART:

En estado de reposo (*Idle*), la línea de transmisión se mantiene en nivel lógico alto (**1**). Una trama estándar de datos está compuesta por los siguientes elementos:

1. **Bit de Inicio (*Start Bit*):** Transición de nivel alto (**1**) a nivel bajo (**0**) durante un período de bit. Notifica al receptor que una nueva trama está comenzando.
2. **Longitud de Palabra (*Data Bits*):** Bloque de datos útil, transmitido habitualmente empezando por el bit menos significativo (*LSB first*). Suele configurarse entre 5 y 9 bits, siendo 8 bits el valor más común.
3. **Bit de Paridad (*Parity Bit, opcional*):** Bit añadido al final de los datos para la detección elemental de errores:
 - **Paridad Par (*Even*):** Se ajusta a 1 o 0 para que el número total de unos en la palabra de datos (incluyendo el bit de paridad) sea par.
 - **Paridad Impar (*Odd*):** Se ajusta para que el número total de unos sea impar.
4. **Bits de Parada (*Stop Bits*):** Uno o varios bits en nivel lógico alto (**1**) al final de la trama que garantizan un tiempo de resincronización mínimo para la línea antes de la siguiente transmisión.

Diagrama de Tiempos Típico:

En la Figura 16 se muestra la representación gráfica de una trama serie de 8 bits de datos con paridad y 1 bit de stop:

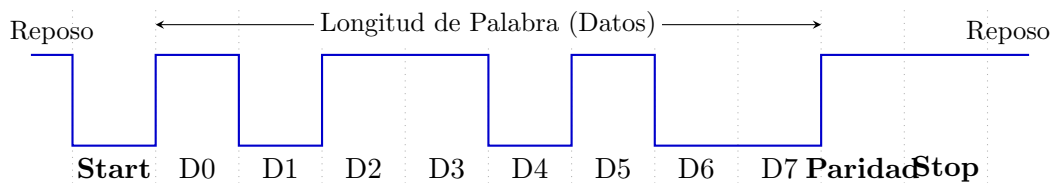


Figura 16: Cronograma de una trama de datos UART de 8 bits con paridad y 1 bit de parada.

4.2.2. UART en el STM32F723IE

En el procesador de la placa, se disponen de varios periféricos de UART diferentes, numerados como USARTx, siendo la x el número de UART. La UART dispone de varios registros de control, otros de configuración, y finalmente registros para la recepción/envío de datos. Todos ellos se describen en [rm0431 27.8](#). Las direcciones base de los periféricos USARTx se encuentran en [rm0431 1.6.2](#). Cómo hacer la transmisión en sí se describe en [rm0431 27.5.2](#).

¿Ahora, qué USARTx utilizamos? en [um2140 5.13](#) nos dice que la USART6 está conectada directamente a la salida desde ST-Link (El depurador que venimos utilizando hasta ahora para conectar con la placa). Por tanto, deberemos configurar USART6. Los pines donde se conecta (que habrá que activar) se pueden ver en [ds11853 Table 10](#). En concreto, vemos que son el pin PC6 para USART6-TX y PC7 para USART6-RX. (De momento solo usamos el TX para transmitir). Para que por ese pin salga la UART (Y no el valor genérico que ponemos en el puerto de salida del GPIO), deberemos configurar la función alternativa 8 como indica [ds11853 Table 12](#), que conecta la USART6 al puerto C.

4.3. Desarrollo de la práctica

Para el desarrollo de esta práctica hay que extender el BSP introduciendo las funciones y definiciones necesarias de la UART, y posteriormente controlarla para emitir datos hacia el ordenador. La documentación necesaria puede verse en [rm0431 27](#). Los pasos a seguir son los siguientes:

1. Según [ds11853 Table 10](#), el pin PC6 es quien saca al exterior la UART. Configúralo en modo de función alternativa en [MODER](#), tipo push-pull en [OTYPER](#), pull-up en [PUPDR](#), velocidad alta en [OSPEEDR](#), y configura la función alternativa en [AFR](#) para que sea la 8 (como indica [ds11853 Table 12](#)). No te olvides de activar el puerto C a través de [AHB1ENR](#).

```
1 RCC_AHB1ENR->bits.GPIOCEN = 1;
2 GPIO_MODE_SET(GPIOC, 6, GPIO_MODE_ALT);
3 //... completar
```

2. Ahora le toca el turno a la USART6 como nos indica [um2140 5.13](#). Actívala desde [APB2ENR](#) lo primero.

```
1 //en bsp.h
2 #define RCC_APB2EN_BASE_ADDR 0x40023844
3 //Structure for Reset Control peripheral register APB1
4 typedef union {
5     volatile uint32_t reg;
6
7     // 2. Access individual bits or bit-groups for the register above
```



```

8      struct {
9          volatile uint32_t TIM1EN      : 1;  // Bit 0:
10         volatile uint32_t TIM8EN      : 1;  // Bit 1:
11         volatile uint32_t RES1        : 2;  // Bit 2-3:
12         //... completar
13     } bits;
14 } RCC_APB2R_TypeDef;
15 #define RCC_APB2ENR      ((RCC_APB2R_TypeDef *) RCC_APB2EN_BASE_ADDR)
16
17
18 //en main.c
19 RCC_APB2ENR->bits.USART6EN = 1;          //enable UART clock

```

3. Configura la UART desde los registros **CR1** y **CR2**. Activa el periférico, configura la longitud de palabra a 8 bits, 1 bit de stop y sin paridad, pone por último el oversampling a 16.
4. Configura la velocidad de la UART desde **BRR**. Tendrás que ponerle el valor del reloj del sistema (16MHz) entre la velocidad de la uart (115200).
5. Por último, activa el modo de transmisión desde **CR1**.

```

1  // UART STOP BITS
2  typedef enum {
3      UART_STOP_ONEBIT = 0x00,
4      UART_STOP_HALFBIT = 0x01,
5      UART_STOP_TWOBIT = 0x02,
6      UART_STOP_ONEHALFBIT = 0x03
7  } UART_Stop_t;
8  #define UART_STOP_MASK 0b11
9  #define UART_STOP_SHIFT 12
10
11 static inline void UART_STOPBIT_SET(volatile USART_TypeDef *USARTx,
12     UART_Stop_t stopbits) {
13     //... completar configurar el registro CR2 con los bits de stop
14 }
15
16 // UART ENABLE
17 typedef enum {
18     UART_DISABLE = 0x00,
19     UART_ENABLE = 0x01
20 } UART_Enable_t;
21 #define UART_ENABLE_MASK 0b1
22 #define UART_ENABLE_SHIFT 0
23
24 static inline void UART_ENABLE_SET(volatile USART_TypeDef *USARTx,
25     UART_Enable_t enable) {
26     //... completar configurar el registro CR1 con el bit de enable
27 }
28
29 // UART WORD LENGTH
30 typedef enum {
31     UART_WORD_8B = 0x00,
32     UART_WORD_9B = 0x01,
33     UART_WORD_7B = 0x02
34 } UART_WordLength_t;
35 #define UART_WORD_MASK 0b1
36 #define UART_WORD_SHIFT 12
37
38 static inline void UART_WORDLENGTH_SET(volatile USART_TypeDef *USARTx,
39     UART_WordLength_t length) {
40     //... completar escribir longitud en registro CR1

```

```

38 }
39
40 //... completar todas estas otras funciones auxiliares
41 static inline void UART_PARITY_SET(volatile USART_TypeDef *USARTx,
    UART_Parity_t parity);
42 static inline void UART_OVERSAMPLING_SET(volatile USART_TypeDef *USARTx,
    UART_Oversampling_t over);
43 static inline void UART_TX_MODE_SET(volatile USART_TypeDef *USARTx,
    UART_TX_Mode_t mode);

```

6. Cuando quieras transmitir un byte, espera primero a que quede libre el transmisor (aparece un 1 en el bit TC del registro **ISR**). A continuación escribe el dato en el registro **TDR**, la UART lo transmitirá solo.

```

1 //... en bsp.h
2 static inline void UART_TransmitByte(volatile USART_TypeDef *USARTx,
    uint8_t data)
3 {
4     while (!(USARTx->ISR & (1U << 7U))); //Wait until we can push data (TXE
        bit enabled)
5     USARTx->TDR = data; //push data (this clears TXE bit)
6 }
7
8
9 //... en main.c
10 UART_TransmitByte(USART6, 'X');
11 UART_TransmitByte(USART6, 'Y');
12 //...

```

Para ver ahora los mensajes desde el ordenador, utilizaremos la utilidad **picocom** (o cualquier otra terminal de vuestro gusto). Desde la máquina virtual, abrir una terminal y ejecutar el comando:

```

1 picocom /dev/ttyACM0 -b 115200 -d 8 -y n -p 1

```

NOTA: El nombre del puerto puede no ser necesariamente **ttyACM0**, aunque en linux al menos es el valor por defecto. En Windows, por ejemplo, suele llamarse **COMx**, siendo x un número arbitrario.

4.4. Para hacer más

Averigua cómo hacer transferencias en modo DMA para poder seguir ejecutando código mientras la UART hace su trabajo “sola”.

5. Práctica 4: Entrada / Salida por interrupciones - UART RX y botones

5.1. Objetivos

- Aprender qué son las interrupciones, cómo funcionan, y sus características.
- Programar el controlador de interrupciones para la recepción de datos con UART.
- Programar el controlador de interrupciones para entrada con botón.

5.2. Fundamentos teóricos

5.2.1. Configuración de las interrupciones en el procesador

Para responder de manera rápida y consistente a eventos en un microcontrolador, es buena idea utilizar una *interrupción hardware* para ejecutar un pequeño fragmento de código al detectar dichos eventos. En esta práctica, vamos a ver cómo funciona el controlador de interrupciones en STM32, que puede configurarse para ejecutar ciertas funciones al ocurrir determinados eventos.

Cada procesador tiene su propia manera de tratar interrupciones. En la familia STM32, y en concreto de **STM32F723IE**, se utilizan dos periféricos diferentes para su gestión:

- **NVIC** : El (*Nested Vectored Interrupt Controller*) [pm0253 4.2](#) es el gestor de interrupciones interno del procesador Cortex-M7, núcleo dentro del **STM32F723IE**. Permite la programación de hasta 240 interrupciones, con control de prioridad en cada una de ellas. En el caso de nuestro procesador, **STM32F723IE**, están disponibles unas 100 ([rm0431 9.1.2](#)), aunque utilizaremos nada más que un par de ellas.
- **EXTI** : (*EXtended Interrupt Controller*). Su propósito es extender el número de interrupciones disponibles a los programadores a través de eventos externos al propio procesador. Cada periférico interno tiene sus propias líneas de interrupción en el **NVIC**, como puede verse en [rm0431 9.1.2](#), sin embargo, para los eventos en pines de E/S (a través de GPIO), la configuración de la interrupción se hace desde el **EXTI**.

En la familia STM32, los pines GPIO están agrupados (PA0, PB0, PC0...). El periférico **EXTI** no tiene una línea directa para cada pin individual de la placa, sino que usa líneas compartidas (la línea **EXTI0** gestiona PA0, PB0, PC0, etc. a través de un multiplexor en el **SYSCFG**). Por tanto, no se pueden usar, por ejemplo, PA0 y PB0 como interrupciones independientes simultáneamente.

Adicionalmente, los periféricos externos, aunque estén conectados a las líneas de interrupción, deben ser configurados para generar las señales de interrupción necesarias.

En resumen, estos procesadores tienen interrupciones internas a través del **NVIC**, que hay que activar, además de realizar la configuración de los periféricos y, algunos casos como el GPIO, configurar **EXTI** para recibir sus interrupciones. En esta práctica, configuraremos las interrupciones de **UART** directamente desde **NVIC**, y las de **GPIO** utilizando también **EXTI**. Podemos ver un diagrama de las interconexiones entre todo en la Figura 17.

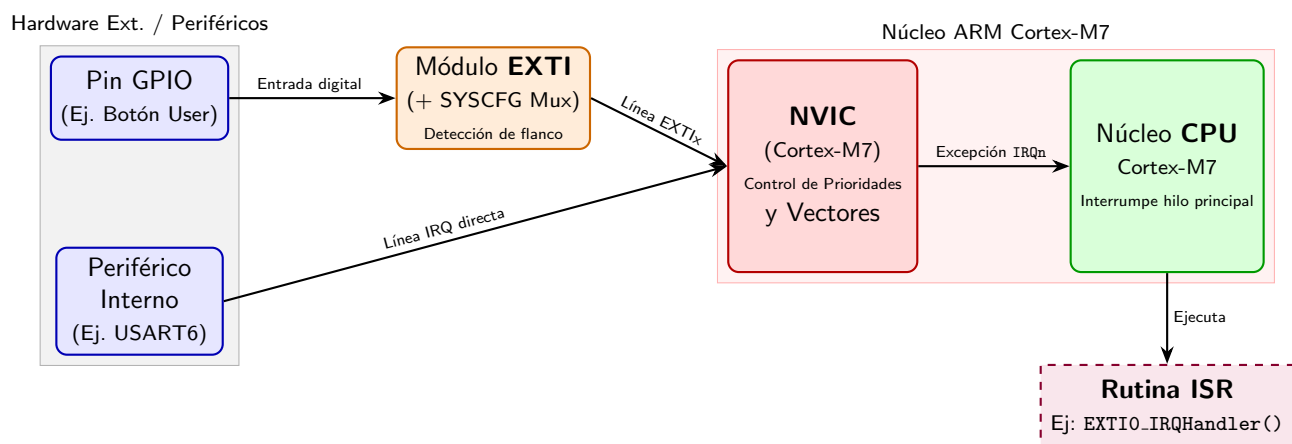


Figura 17: Flujo de interrupciones en el microcontrolador STM32F723E.

5.2.2. Gestión de interrupciones mediante funciones

Cuando se produce una interrupción, el procesador debe pausar su ejecución y saltar a la dirección de memoria donde se encuentra la rutina de respuesta o ISR (Interrupt Service Routine). Como la ubicación física de estas funciones en la memoria Flash no es fija (depende de la compilación), el procesador utiliza una Tabla de Vectores de Interrupción (Vector Table).

Esta tabla es un array de punteros ubicado al inicio de la memoria (por defecto desde `0x00000000`), donde cada posición está reservada para un evento concreto:

- La posición `0x00000000` almacena el valor inicial del puntero de pila (MSP).
- La dirección `0x00000004` contiene el vector de Reset.
- Direcciones posteriores contienen los vectores o punteros a las rutinas de interrupción, como por ejemplo `0x0000015C` que contiene el puntero a la rutina de la `USART6` ([rm0431 9.1.2](#)).

Para evitarnos la tarea de configurar esta tabla manualmente, el código de inicio del procesador (`Startup/startup_stm32f723ieqx.s`) la genera automáticamente durante el arranque. Por defecto, todas las entradas de la tabla apuntan a un bucle infinito genérico (`Default_Handler`) mediante el atributo `.weak`. Si definimos en nuestro código C una función con el nombre exacto de la interrupción (por ejemplo, `USART6_IRQHandler`), el enlazador (linker) la sobrescribirá automáticamente, redirigiendo la tabla hacia nuestra rutina. Podemos ver este proceso en Figura 18.

5.3. Desarrollo de la práctica

Como en prácticas anteriores, el objetivo es seguir extendiendo nuestro BSP con las funciones necesarias como para controlar los periféricos y funcionalidades que utilizaremos en esta práctica. En este caso, debemos hacer varias cosas:

1. Apúntate las direcciones base de los periféricos `NVIC` en [pm0253 4.2](#), y `EXTI` y `SYSCFG` en [rm0431 1.6.2](#), crea además las estructuras de datos con las que accederás a los registros. Básate, respectivamente, en [pm0253 4.2](#) para `NVIC` (Ten en cuenta que los

1. Tabla de Vectores (Flash / RAM)

Offset / Dir.	Contenido (Vector)
0x00000000	Stack Pointer
0x00000004	Reset_Handler
...	...
0x00000058	EXTIO_IRQHandler
...	...
0x0000015C	USART6_IRQHandler
...	...

2. Funciones en Flash (.text)

Dirección	Código / Función
0x08000100	Reset_Handler()
0x08000250	Default_Handler()
...	...
0x08001F10	USART6_IRQHandler()
0x08002F4C	EXTIO_IRQHandler()

Puntero

Por defecto (.weak)

Redirigido por C

Redefinición en C:

Al escribir void USART6_IRQHandler(void) en C, el linker asigna la dirección real de tu función en la tabla.

Atributo .weak en Startup:

Si el usuario NO define la función en C, la tabla apunta al Default_Handler (bucle infinito).

Figura 18: Mapeo de la Tabla de Vectores de Interrupción a la memoria del programa.

registros no están consecutivos), en [rm0431 10.7.7](#) para **EXTI**, y en [rm0431 7.2.8](#) para **SYSCFG**. Crea además las variables para manejarlos. Puedes basarte en el siguiente código:

```

1 #define EXTI_BASE_ADDR      0x40013C00UL
2 //... completar para NVIC y SYSCFG
3
4 typedef struct
5 {
6     volatile uint32_t ISER[8U];           /*!< Offset: 0x000 (R/W)
7         Interrupt Set Enable Register */
8     uint32_t RESERVED0[24U];
9     //... completar
10 } NVIC_TypeDef;
11
12 typedef struct
13 {
14     volatile uint32_t MEMRMP;             /*!< SYSCFG memory remap register,
15                                         Address offset: 0x00 */
16     //... completar
17 } SYSCFG_TypeDef;
18
19 typedef struct
20 {
21     volatile uint32_t IMR;                /*!< EXTI Interrupt mask register,
22                                         Address offset: 0x00 */
23     //... completar
24 } EXTI_TypeDef;
25
26 #define EXTI                  ((EXTI_TypeDef *) EXTI_BASE_ADDR)
27 //... completar para NVIC y SYSCFG

```

- Debemos activar el puerto y pin correspondiente al botón. Revisando la documentación, en [um2140 Table 5](#) nos dice que el botón está enchufado a la función wake-up del procesador, y en [um2140 Table 19](#) que dicha función SYS_WKUP1 está en el PA0. Por tanto, lo primero es configurar este pin como entrada, de tipo *push-pull* con el *pulldown* activado, y a velocidad alta. No te olvides de activar también el reloj de GPIOA a través de [AHB1ENR](#).

3. A continuación, hay que activar también el pin de transmisión de la UART. En [um2140 Table 19](#) podemos observar que la función `USART6_RX` está conectada al pin `PC7`. Tenemos que configurarlo en modo *alternativo* con la función 8 según nos indica [ds11853 Table 12](#). Adicionalmente, configurarlo en modo *push-pull* con resistencia de *pullup* y velocidad alta.
4. Finalmente, configuramos la `UART` para que también funcione en modo recepción y tenga activas las interrupciones de recepción. Ambas cosas se pueden hacer desde el registro `CR1` ([rm0431 27.8.1](#)). Crea funciones en el BSP y define las constantes necesarias. Puedes hacer algo de este estilo:

```

1 //en main.c
2 UART_RX_MODE_SET(USART6, UART_RX_ENABLE);
3 UART_RX_INT_SET(USART6, UART_RX_INT_ENABLE);
4
5
6 //en bsp.h
7 // RECEIVE enable
8 typedef enum {
9     UART_RX_DISABLE = 0x00,
10    UART_RX_ENABLE  = 0x01
11 } UART_RX_Mode_t;
12 #define UART_RX_MODE_MASK 0b1
13 #define UART_RX_MODE_SHIFT 2
14
15 static inline void UART_RX_MODE_SET(volatile USART_TypeDef *USARTx,
16    UART_RX_Mode_t mode) {
17     //... completar tocar el bit adecuado de CR1
18 }
19
20 //... completar hacer una función similar para RX INT enable

```

5. Ahora ya podemos comenzar a activar las interrupciones. Lo primero es activar el reloj para la configuración del sistema en `APB2ENR` ([rm0431 5.3.14](#)), para lo cual previamente deberemos definir los campos del registro.

```

1 //en bsp.h
2 #define RCC_APB2EN_BASE_ADDR 0x40023844 //APB2 clock control register
3
4 //Structure for Reset Control peripheral register APB1
5 typedef union {
6     volatile uint32_t reg;
7
8     // 2. Access individual bits or bit-groups for the register above
9     struct {
10         volatile uint32_t TIM1EN      : 1; // Bit 0:
11         volatile uint32_t TIM8EN      : 1; // Bit 1:
12         volatile uint32_t RES1        : 2; // Bit 2-3:
13         //... completar
14     } bits;
15 } RCC_APB2R_TypeDef;
16
17 #define RCC_APB2ENR ((RCC_APB2R_TypeDef *) RCC_APB2EN_BASE_ADDR)

```

A continuación, debemos configurar la interrupción externa `EINT0` (que multiplexa todos los pines 0) para que trabaje con el puerto A (recordemos que el botón está en `PA0`). Esto se hace desde el registro `EXTICR` de `SYSCFG` ([rm0431 7.2.3](#)). Finalmente, hay que activar la interrupción externa 0 desde `EXTI` a través del registro `IMR` ([rm0431 10.9.1](#)) en modo *rising trigger* en el registro `RTSR` ([rm0431 10.9.3](#)).

```

1 //... completar en bsp.h crear y rellenar estas funciones

```

```

2 static inline void SYSCFG_EINT_MAP_PORT(uint32_t eintno, uint32_t portno) {
3     //... completar modificar SYSCFG->EXTICR
4 }
5
6 static inline void EXTI_ENABLE_RISING_TRIGGER(uint32_t eintno, uint32_t
7     enable) {
8     //... completar activar/desactivar EXTI->RTSR
9 }
10
11 static inline void EXTI_UNMASK_INTERRUPT(uint32_t eintno, uint32_t unmask)
12 {
13     //... completar activar/desactivar EXTI->IMR
14 }
15
16 //en main.c
17 RCC_APB2ENR->bits.SYSCFGEN = 1;
18 //Activar EINT0 PUERTO A
19 SYSCFG_EINT_MAP_PORT(0, 0); //port a (0) to eint0
20 EXTI_ENABLE_RISING_TRIGGER(0, 1); //enable on port a
21 EXTI_UNMASK_INTERRUPT(0, 1); //unmask on port a

```

6. Ya tendríamos con esto el pin PA0 generando una interrupción en EXTI0, pero el procesador aún no responde a estas interrupciones. Adicionalmente, necesitamos activar la interrupción de la **UART**. Ambas cosas se hacen desde el **NVIC** en el registro **ISER** (**pm0253 4.2.2**). Para saber qué bits activar, debemos consultar la numeración de las líneas de interrupción en **rm0431 9**.

```

1 //en bsp.h
2 //... completar buscar los números de línea
3 #define EXTI0_LINE_IRQn
4 #define USART6_INT_IRQn
5
6 static inline void NVIC_ENABLE_INT(uint32_t line) {
7     //... completar modificar el registro NVIC->ISER
8 }
9
10 //en main.h
11 NVIC_ENABLE_INT(EXTI0_LINE_IRQn);
12 NVIC_ENABLE_INT(USART6_INT_IRQn);

```

7. Por último, necesitamos crear dos funciones que respondan a sendas interrupciones. En este caso, la función **USART6_IRQHandler** que responda a la **USART6**, y **EXTI0_IRQHandler** que responda al botón.
8. En el caso de la primera, **USART6_IRQHandler**, deberemos comprobar primero que la interrupción ha sido generada por recepción, consultando el bit **RXNE** del registro **ISR** de la **UART** (**rm0431 27.8.8**). Si es así, podremos leer el registro **RDR** (**rm0431 27.8.10**) para consultar el dato recibido, lo cual a su vez limpiará la interrupción. Dentro de la interrupción, para darle funcionalidad, podemos por ejemplo enviar de vuelta el dato leído al ordenador.

```

1 void USART6_IRQHandler(void)
2 {
3     //... completar comprobar si el flag de interrupción está activo
4     if () {
5         //... completar leer el byte
6     }
7 }

```

9. En el caso de la segunda `EXTIO_IRQHandler`, el proceso es diferente. Primero, debemos comprobar que la interrupción está pendiente desde el registro `PR` de `EXTI` ([rm0431 10.9.6](#)). A continuación, habrá que borrar dicho bit, escribiendo de nuevo en el registro, como indica la documentación. En esta función, por ejemplo, se puede poner como funcionalidad el dar la vuelta al LED que ya teníamos configurado anteriormente, para comprobar que funciona.

```
1 void EXTI0_IRQHandler(void)
2 {
3     //... comprobar en EXTI->PR si la línea 0 generó interrupción
4     if () {
5         //... limpiar línea 0 escribiendo la posición 0 de EXTI->PR
6         //... completar funcionalidad
7     }
8 }
```

5.4. Para hacer más

Intenta cambiar “en caliente” la función que gestiona una interrupción, por ejemplo la del botón, modificando el puntero de las direcciones de memoria a las que salta el `NVIC`.

6. Práctica 5: Configuración de Timers

En esta práctica vamos a introducir el concepto de temporizador. Son unos periféricos del procesador que permiten, cada cierto tiempo, generar interrupciones y, con ello, disparar funcionalidades concretas que hayamos programado. En este caso programaremos uno para hacer parpadear un LED, y otro diferente para utilizar como *debouncer* del botón.

6.1. Objetivos

- Entender el funcionamiento de los temporizadores internos del sistema.
- Interconectar interrupciones de varios periféricos para lograr funcionalidades avanzadas.

6.2. Fundamentos teóricos

Un temporizador o *timer* en un microcontrolador es un periférico hardware que, en esencia, es un contador digital. La cuenta se realiza en función de una señal de reloj de entrada, y al llegar a ciertos valores (configurables) dispara interrupciones.

Es capaz de medir intervalos de tiempo de forma independiente a la ejecución de código en la CPU, liberándola pues para que haga otras tareas en lugar de esperas activas a ciertos eventos. Los temporizadores suelen contar con varios registros para su control, entre otros:

- **PSC** o *prescaler*: Un registro que sirve para dividir la frecuencia de entrada. Normalmente, la frecuencia del procesador están en el orden de MHz. El prescaler suele dividirlo al orden de KHz o incluso menos, según las necesidades de temporización.
- **CNT** o *counter*: El registro que lleva la cuenta actual del timer. Suma 1 cada vez que la señal de reloj, dividida entre el factor del prescaler, se activa. Su frecuencia de actualización f_{CNT} es por tanto:

$$f_{CNT} = \frac{f_{CLK}}{\text{PSC} + 1}$$

- **ARR** o *auto-reload*: Valor máximo de conteo. Cuando **CNT** alcanza este valor, se resetea y genera un evento de interrupción que podremos controlar. El periodo de evento T_{UEV} es por tanto:

$$T_{UEV} = \frac{\text{ARR} + 1}{f_{CNT}} = \frac{(\text{PSC} + 1)(\text{ARR} + 1)}{f_{CLK}}$$

- **CCR** o *capture compare*: Registros que permiten diversas configuraciones de comparación con CNT (mayor que, menor, igual ...) para medir la duración de señales externas o generar formas de onda PWM, entre otras funcionalidades. En ocasiones también permiten interrupciones adicionales a la de **ARR**.

Podemos ver un diagrama de funcionamiento en la Figura 19.

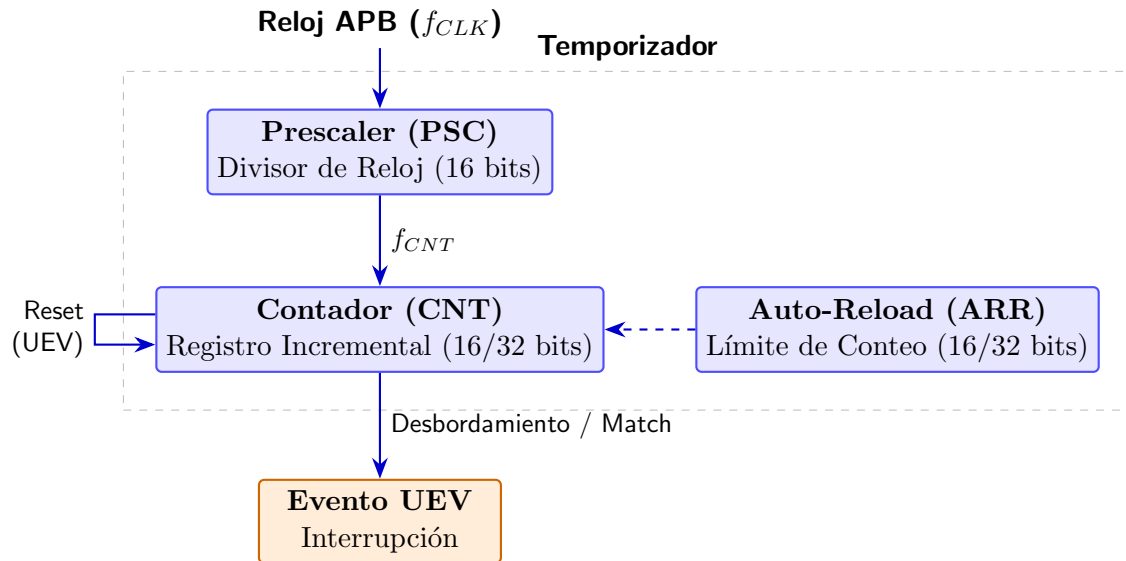


Figura 19: Diagrama de funcionamiento simplificado para los timers

6.3. Desarrollo de la práctica

Lo primero, como siempre, es ir añadiendo a nuestro `bsp.h` lo necesario para poder gestionar los timers. Vamos a utilizar dos, `TIM2` y `TIM3`. Consulta [rm0431 1.6](#) para ver las direcciones base de memoria, y [rm0431 19.4](#) para los registros de los temporizadores que permitan crear el `typedef` adecuado.

```

1 //... completar
2 #define TIM2_BASE_ADDR
3 #define TIM3_BASE_ADDR
4
5 //Structure for Timers (TIM2 / TIM3)
6 typedef struct
7 {
8     volatile uint32_t CR1;          /*!< TIM control register 1,
9         Address offset: 0x00 */
10    volatile uint32_t CR2;          /*!< TIM control register 2,
11        Address offset: 0x04 */
12    //... completar
13 } TIM_TypeDef;
14
15 #define TIM2 ((TIM_TypeDef *) TIM2_BASE_ADDR)
16 #define TIM3 ((TIM_TypeDef *) TIM3_BASE_ADDR)

```

A continuación tenemos que crear las funciones necesarias para el control de los timers:

```

1 //funciones de configuración
2 //... activa en CR1 el bit correspondiente para activar el contador
3 static inline void TIM_ENABLE_COUNTER(volatile TIM_TypeDef *TIMx, uint32_t
4     enable);
5 //... activa el modo oneshot desde CR1
6 static inline void TIM_ENABLE_ONESHOT(volatile TIM_TypeDef *TIMx, uint32_t
7     enable);
8 //... configura el registro PSC con el valor de preescalado
9 static inline void TIM_SET_PRESCALER(volatile TIM_TypeDef *TIMx, uint32_t value)
10 ;
11 //... configura el registro ARR con el valor de autoreload
12 static inline void TIM_SET_AUTO_RELOAD(volatile TIM_TypeDef *TIMx, uint32_t

```

```

    value);
10 //... activa las interrupciones en el registro DIER
11 static inline void TIM_ENABLE_INT(volatile TIM_TypeDef *TIMx);
12 //funciones de uso
13 //... borra el flag pendiente del registro SR
14 static inline void TIM_CLEAR_FLAG(volatile TIM_TypeDef *TIMx);
15 //... comprueba si hay flag pendiente en el registro SR
16 static inline uint32_t TIM_CHECK_FLAG(volatile TIM_TypeDef *TIMx);

```

También es necesario crear una función que nos permita inicializar el timer, llamando a las anteriores de configuración:

```

1 //congiura prescaler, autoreload, activa counter y oneshot si es necesario y
  activa interrupciones
2 static void TIM_INIT(TIM_TypeDef *TIMx, uint32_t prescaler, uint32_t autoreload
  , uint32_t enable_counter, uint32_t enable_oneshot);

```

Y por último tendremos que activar los bits correspondientes en **APB1ENR** ([rm0431 5.3.13](#)), y adicionalmente activar en **NVIC** ([rm0431 9.1.12](#)) las líneas de interrupción adecuadas:

```

1 RCC_APB1ENR->bits.TIM2EN = 1;
2 RCC_APB1ENR->bits.TIM3EN = 1;
3 TIM_INIT(TIM2, 15999, 500, 1, 0); //500 milisegundos, activo
4 TIM_INIT(TIM3, 15999, 10, 0, 1); //10 milisegundos, inactivo, oneshot
5 //Enable interrupts
6 NVIC_ENABLE_INT(TIM2_IRQn);
7 NVIC_ENABLE_INT(TIM3_IRQn);

```

Ahora podemos, por fin, hacer uso de las funciones de los timers para ver su efecto:

```

1 // Handles Task 1: Blink LED1 (Green) each 0.5 seconds
2 void TIM2_IRQHandler(void) {
3     //... completar comprobar que el flag está activo
4     if () {
5         //... completar borrar el flag
6         //... completar funcionalidad deseada
7     }
8 }
9
10 void TIM3_IRQHandler(void) {
11     //... completar comprobar que el flag está activo
12     if () {
13         //... completar borrar el flag
14         //... completar funcionalidad deseada
15     }
16 }

```

La funcionalidad que queremos se muestra en la Figura 20

- **TIM2** enciende o apaga el led verde (PB1) cada vez que se dispara. Para ello, asegúrate de haber activado dicho puerto y pin como **GPIO**.
- **TIM3** sirve como *debouncer* para el botón (en PA0). Para ello debemos conseguir que, al pulsarse el botón, se inhabilite durante el tiempo que tarde el timer en dispararse. Si tras ese tiempo el botón sigue pulsado, contaremos que es una pulsación de verdad. En caso contrario, lo omitiremos. En ambos casos, al terminar el temporizador volveremos a activar las interrupciones del botón. Como está en modo *oneshot*, cada vez que lo activemos se disparará solo una vez, evitando problemas.

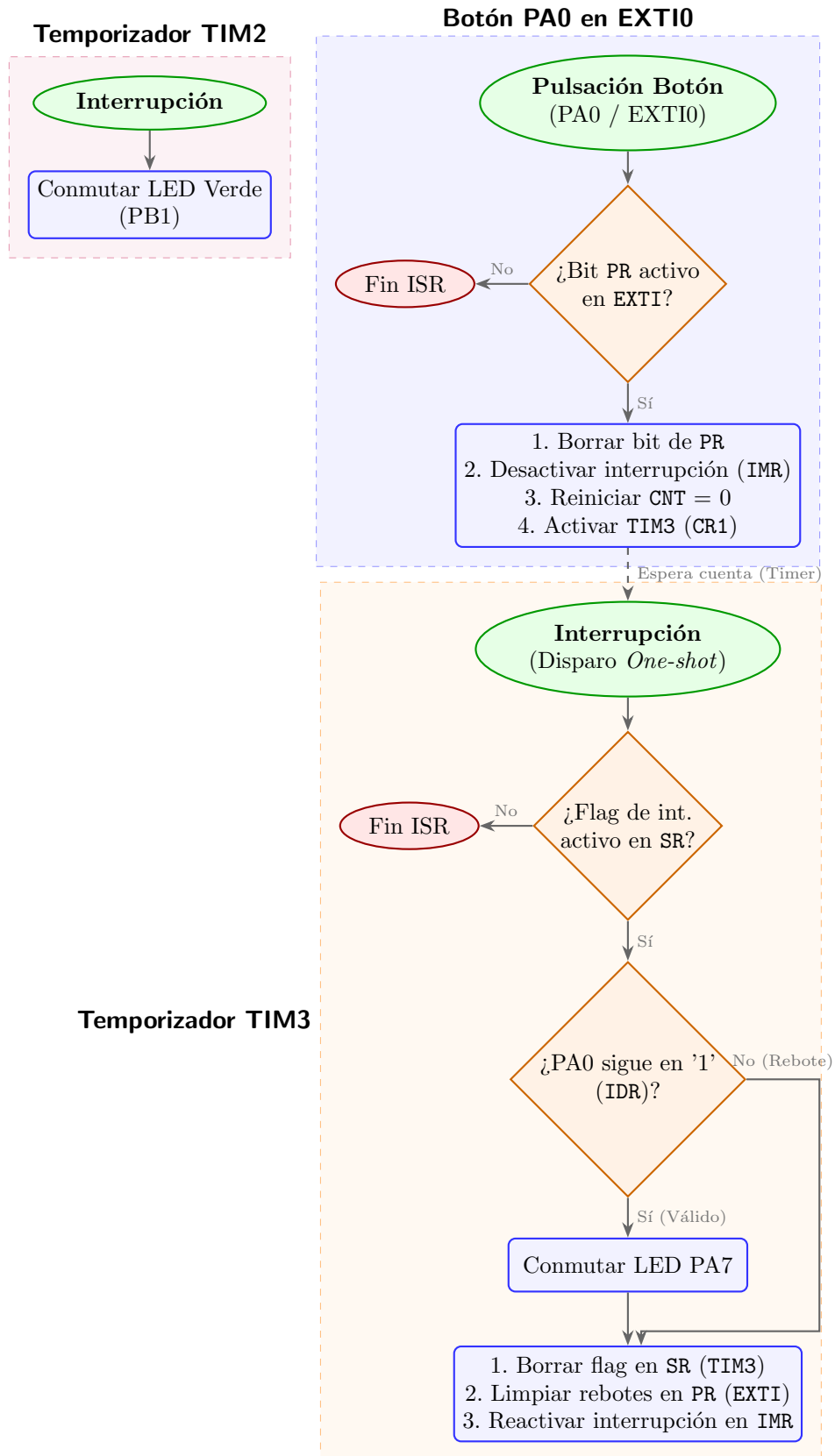


Figura 20: Diagrama de flujo funcional de las interrupciones.

Para el botón, debemos comprobar tras la activación que el bit correspondiente de **PR** está activo en **EXTI** ([rm0431 10.9.6](#)). Si es así, lo borramos y a continuación desactivamos la interrupción desde el registro **IMR** ([rm0431 10.9.1](#)). Finalmente activamos el temporizador, reseteando **CNT** a cero y activándolo desde **CR1** ([rm0431 19.4.1](#)).

```

1 void EXTI0_IRQHandler(void)
2 {
3     //... comprobar si la interrupción está pendiente
4     if () {
5         //... borrar bit pendiente
6         //... desactivar interrupciones de botón
7         //... resetear cuenta de timer
8         //... activar timer
9     }
10 }

```

En cuanto al timer, debemos en primer lugar comprobar y borrar si es necesario el flag de interrupción desde **SR** ([rm0431 19.4.5](#)). A continuación, y si el botón sigue pulsado (podemos comprobarlo desde el registro **IDR** del **GPIO** ([rm0431 6.4.5](#))), efectuar la función deseada (por ejemplo hacer parpadear el LED conectado a PA7). Tras ello, borraremos interrupciones pendientes del botón (por si se hubiera quedado pillado con rebotes) desde **PR** de **EXTI** ([rm0431 10.9.6](#)), y re-activaremos las interrupciones de botón en **IMR** ([rm0431 10.9.1](#)). Una cuestión importante es que, gracias a haber configurado el temporizador en modo *oneshot*, no es necesario desactivar sus propias interrupciones ya que no volverá a dispararse hasta reiniciar la cuenta.

```

1 // Debounce Lockout Timer Handler (Fires 10ms after initial edge)
2 void TIM3_IRQHandler(void) {
3     //... comprobar flag de interrupción
4     if () {
5         //... borrar flag de interrupción
6         //... verificar que sigue pulsado PA0
7         if () {
8             //... funcionalidad... Toggle led de PA7
9         }
10        //... borrar interrupciones pendientes para EXTI0 (botón)
11        //... reactivar interrupciones para EXTI0
12    }
13 }

```

6.4. Para hacer más

Hay un “temporizador” especial llamado **RTC** (*Real Time Clock*) capaz de llevar la cuenta de la hora y fecha actual de forma precisa. Con ello, además, puede generar interrupciones a largo plazo. Se utiliza principalmente para realizar esperas largas donde el procesador está dormido mientras espera la interrupción. Consulta la documentación y utilízalo para hacer parpadear el led azul (PA5) cada 5 segundos.